Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №10

з дисципліни

«Технології створення програмних продуктів»

ШАБЛОН COMMAND

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

Завдання: Створіть прототип текстового редактора, який підтримує базові операції редагування, такі як вставка, видалення, копіювання та відміна. Використовуйте шаблон “Command” для реалізації цих функцій, щоб користувачі могли легко відміняти та повторювати свої дії.

Код реалізації:

```
using System;
using System.Collections.Generic;

namespace CommandTask
{
    public class Document
    {
        public string Text { get; set; } = "";
        public string Clipboard { get; set; } = "";

        public void Print()
        {
            Console.WriteLine($"[Текст документа]: {Text}");
        }
    }

    public abstract class Command // Command
    {
        protected Document _document;

        public Command(Document document)
        {
            _document = document;
        }

        public abstract void Execute();
        public abstract void Undo();
    }

    public class InsertCommand : Command // ConcreteCommand
    {
        private string _insertedText;

        public InsertCommand(Document doc, string text) : base(doc)
        {
            _insertedText = text;
        }

        public override void Execute()
        {
            _document.Text += _insertedText;
        }

        public override void Undo()
        {
            _document.Text = _document.Text.Substring(0, _document.Text.Length -
_insertedText.Length);
        }
    }

    public class DeleteCommand : Command
    {
        private int _length;
        private string _deletedText;

        public DeleteCommand(Document doc, int length) : base(doc)
        {
            _length = length;
```

```
        }

        public override void Execute()
        {
            if (_document.Text.Length >= _length)
            {
                _deletedText = _document.Text.Substring(_document.Text.Length -
_length);
                _document.Text = _document.Text.Substring(0, _document.Text.Length -
_length);
            }
        }

        public override void Undo()
        {
            if (_deletedText != null)
            {
                _document.Text += _deletedText;
            }
        }
    }

    public class CopyCommand : Command
    {
        private string _previousClipboard;

        public CopyCommand(Document doc) : base(doc) { }

        public override void Execute()
        {
            _previousClipboard = _document.Clipboard;
            _document.Clipboard = _document.Text;
            Console.WriteLine($"[Система]: Текст скопійовано до буфера обміну.");
        }

        public override void Undo()
        {
            _document.Clipboard = _previousClipboard;
        }
    }

    public class TextEditor
    {
        private Document _document = new Document();

        private Stack<Command> _undoStack = new Stack<Command>();
        private Stack<Command> _redoStack = new Stack<Command>();

        public void ExecuteCommand(Command command)
        {
            command.Execute();
            _undoStack.Push(command);
            _redoStack.Clear();
        }

        public void Undo()
        {
            if (_undoStack.Count > 0)
            {
                Command command = _undoStack.Pop();
                command.Undo();
                _redoStack.Push(command);
                Console.WriteLine("\n--- Відміна останньої дії (Undo) ---");
            }
        }

        public void Redo()
        {
            if (_redoStack.Count > 0)
```

```
            {
                Command command = _redoStack.Pop();
                command.Execute();
                _undoStack.Push(command);
                Console.WriteLine("\n--- Повторення скасованої дії (Redo) ---");
            }
        }

        public void ShowDocument()
        {
            _document.Print();
        }

        public Document GetDocument() => _document;
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Прототип текстового редактора (Шаблон Command)
===\n");

            TextEditor editor = new TextEditor();

            editor.ExecuteCommand(new InsertCommand(editor.GetDocument(), "Породи котів:
"));
            editor.ExecuteCommand(new InsertCommand(editor.GetDocument(), "Мейн-кун,
Сіамська, Перська, Британська короткошерста, Шотландська капловуха, Сфінкс"));
            editor.ShowDocument();

            editor.ExecuteCommand(new CopyCommand(editor.GetDocument()));

            editor.ExecuteCommand(new DeleteCommand(editor.GetDocument(), 57));
            editor.ShowDocument();

            editor.Undo();
            editor.ShowDocument();

            editor.Redo();
            editor.ShowDocument();

            Console.ReadKey();
        }
    }
}
```

У цьому завданні застосовано поведінковий шаблон Command. Він інкапсулює кожну операцію користувача (вставка, видалення, копіювання) як окремий об'єкт. Це дозволило класу TextEditor (ініціатору) накопичувати історію виконаних команд у структурах типу Stack. Завдяки тому, що всі команди мають метод Undo, ініціатор може легко скасовувати останні дії та повторювати їх знову, забезпечуючи гнучке керування станом документа.

Результат виконання програми:

```
=== Прототип текстового редактора (Шаблон Command) ===

[Текст документа]: Породи котів: Мейн-кун, Сіамська, Перська, Британська короткошерста, Шотландська капловуха, Сфінкс
[Система]: Текст скопійовано до буфера обміну.
[Текст документа]: Породи котів: Мейн-кун, Сіамська, Перська

--- Відміна останньої дії (Undo) ---
[Текст документа]: Породи котів: Мейн-кун, Сіамська, Перська, Британська короткошерста, Шотландська капловуха, Сфінкс

--- Повторення скасованої дії (Redo) ---
[Текст документа]: Породи котів: Мейн-кун, Сіамська, Перська
```

Висновок: під час роботи було успішно застосовано шаблон Command для створення текстового редактора. Шаблон дозволив перетворити кожну операцію (вставка, видалення) на окремий об'єкт. Це усунуло пряму залежність між інтерфейсом та логікою документа , а також дало змогу легко реалізувати функції скасування (Undo) та повторення (Redo) дій користувача через збереження історії команд.